

Unidad 5: Funciones en Python

Introducción a las Funciones

Las funciones en Python son bloques de código que se agrupan bajo un nombre único y se pueden reutilizar múltiples veces. Su principal utilidad radica en la capacidad de agrupar y reutilizar código repetitivo, lo cual facilita la organización y el mantenimiento del código.

¿Qué son las Funciones?

Una función es una agrupación de sentencias que realizan una tarea específica. Las funciones se utilizan para evitar la repetición de código y para dividir un programa en módulos más pequeños y manejables. Esto es especialmente útil en programas grandes, donde la reutilización de código es fundamental para la eficiencia y la claridad del mismo.

Importancia de las Funciones

1. **Modularidad:** Permiten dividir un programa grande en componentes más pequeños y manejables.
 2. **Reutilización de Código:** Se puede reutilizar el mismo bloque de código en diferentes partes del programa sin tener que escribirlo nuevamente.
 3. **Mantenimiento:** Facilitan la corrección de errores y la actualización de código, ya que cualquier cambio realizado en la función se refleja en todas las partes del programa que la utilizan.
-

Creación de Funciones

En Python, una función se define utilizando la palabra clave `def`, seguida del nombre de la función y paréntesis. Dentro de los paréntesis, se pueden definir parámetros que la función puede recibir. Después del nombre y los paréntesis, se incluye un bloque de código indentado que constituye el cuerpo de la función.

Ejemplo de una función básica:

```
'''python
def saludar():
    print('Hola, ¿cómo estás?')
'''
```

Para llamar a esta función, simplemente se utiliza su nombre seguido de paréntesis:

```
```python
saludar()
```

```
```
```

Esto imprimirá:

```
```
```

```
Hola, ¿cómo estás?
```

```
```
```

Parámetros y Argumentos

Las funciones pueden recibir datos, conocidos como parámetros, que les permiten realizar operaciones con esos datos. Los valores que se pasan a las funciones se denominan argumentos.

Ejemplo con parámetros:

```
```python
def saludar_con_nombre(nombre):
 print(f'Hola, {nombre}, ¿cómo estás?')
```

```
```
```

Llamando a la función con un argumento:

```
```python
saludar_con_nombre('Juan')
```

```
```
```

Esto imprimirá:

```
```
```

```
Hola, Juan, ¿cómo estás?
```

```
```
```

Retorno de Valores

Las funciones también pueden devolver datos al programa principal utilizando la palabra clave ``return``. Esto permite que la función realice una operación y proporcione el resultado al llamador de la función.

Ejemplo de función que retorna un valor:*

```
```python
def sumar(a, b):
 return a + b

```
```

Llamando a la función y almacenando el resultado:

```
```python
resultado = sumar(5, 3)
print(resultado)

```
```

Esto imprimirá:

```
```
8
```
```

Conclusión

Las funciones son una herramienta poderosa en Python que permiten agrupar, reutilizar y organizar el código de manera eficiente. Comprender cómo definir, utilizar y retornar valores en funciones es fundamental para escribir programas claros y eficientes.

Sintaxis Básica de una Función

En Python, la definición de una función se realiza utilizando la palabra clave ``def``, seguida del nombre de la función y paréntesis que pueden contener parámetros. A continuación, se presenta un bloque de código indentado que constituye el cuerpo de la función. Los componentes básicos de una función incluyen el nombre de la función, parámetros (opcional), sentencias y un valor de retorno (opcional).

Componentes de una Función

1. **Palabra clave `def`**: Indica el comienzo de la definición de una función.
 2. **Nombre de la función**: Un identificador único que se utiliza para llamar a la función.
 3. **Paréntesis `()`**: Contienen los parámetros de la función. Si no hay parámetros, los paréntesis están vacíos.
 4. **Dos puntos `:`**: Indican el inicio del bloque de código de la función.
 5. **Sentencias**: Bloque de código indentado que define las acciones que realizará la función.
 6. **Valor de retorno**: Utiliza la palabra clave `return` para devolver un valor desde la función al llamador (opcional).
-

Ejemplo de Sintaxis de una Función Básica

```
```python
def nombre_de_la_funcion(parámetro1, parámetro2):
 # Sentencias que realizan alguna operación
 resultado = parámetro1 + parámetro2
 return resultado
```
```

En este ejemplo:

- `def` es la palabra clave que indica el comienzo de una definición de función.
 - `nombre\_de\_la\_funcion` es el nombre de la función.
 - `parámetro1` y `parámetro2` son los parámetros que la función puede recibir.
 - `resultado = parámetro1 + parámetro2` es una sentencia dentro del cuerpo de la función que realiza una operación.
 - `return resultado` devuelve el resultado de la operación al llamador de la función.
-

Ejemplo Práctico

```
```python
def saludar(nombre):
 mensaje = f'Hola, {nombre}!'
 return mensaje
```
```

En este ejemplo:

- ``def`` indica que se está definiendo una función.
- ``saludar`` es el nombre de la función.
- ``nombre`` es un parámetro que la función recibe.
- ``mensaje = f'Hola, {nombre}!'`` es una sentencia que crea un mensaje de saludo.
- ``return mensaje`` devuelve el mensaje al llamador de la función.

Llamada a la función:

```
```python
resultado = saludar('Ana')
print(resultado)

```
```

Esto imprimirá:

```
```
Hola, Ana!

```
```

Definición y Llamada de una Función

La definición de una función establece el bloque de código que se ejecutará cuando se llame a la función. La llamada de una función es la forma en que se invoca la ejecución del bloque de código definido.

Definición de una función:

```
```python
def sumar(a, b):
 return a + b

```
```

Llamada a la función:

```
```python
resultado = sumar(10, 5)
print(resultado)

```
```

Esto imprimirá:

```
'''
15
'''
```

Conclusión

Entender la sintaxis básica para definir funciones en Python es esencial para estructurar el código de manera clara y eficiente. Utilizar la palabra clave `def`, definir parámetros adecuados, escribir sentencias dentro del cuerpo de la función y devolver valores cuando sea necesario son los pasos fundamentales para trabajar con funciones en Python.

Ejemplo de Función Básica

Las funciones en Python permiten agrupar código que puede ser reutilizado múltiples veces. A continuación, se presenta un ejemplo de una función simple llamada `saludar()`, la cual imprime un mensaje de saludo. También se explica cómo llamar a esta función desde el código principal.

Definición de la Función `saludar()`

La función `saludar()` no toma ningún parámetro y su única tarea es imprimir un mensaje en la consola. La definición de esta función es la siguiente:

```
'''python
def saludar():
    print('Estoy saludando desde la función')
'''
```

En esta definición:

- `def` es la palabra clave que indica que se está definiendo una función.
 - `saludar` es el nombre de la función.
 - `()` indica que la función no toma parámetros.
 - `print('Estoy saludando desde la función')` es la sentencia que se ejecuta cuando se llama a la función, la cual imprime el mensaje.
-

Llamada a la Función `saludar()`

Para llamar a la función `saludar()` y ejecutar su código, simplemente se utiliza su nombre seguido de paréntesis:

```
```python
saludar()

```
```

Cuando se llama a la función de esta manera, se ejecuta el bloque de código definido en la función, resultando en la siguiente salida en la consola:

```
```
Estoy saludando desde la función
```
```

Explicación del Proceso

1. **Definición de la Función:** La función se define una vez utilizando la palabra clave `def`. Este proceso crea la función pero no la ejecuta.
2. **Llamada a la Función:** La función se llama utilizando su nombre seguido de paréntesis. Esto ejecuta el bloque de código dentro de la función.

Importancia de la Ejecución en el Código Principal

Llamar a la función desde el código principal es crucial para ejecutar las tareas definidas dentro de la función. Sin la llamada a la función, el código dentro de la función no se ejecutará, y por lo tanto, no se imprimirá el mensaje de saludo.

Ejemplo Completo

```
```python
Definición de la función
def saludar():
 print('Estoy saludando desde la función')

Llamada a la función
saludar()
```
```

```
saludar()
```

```
'''
```

Este código completo define la función `saludar()` y luego la llama para ejecutar su bloque de código, resultando en la impresión del mensaje en la consola.

Conclusión

Definir y llamar a funciones en Python es un proceso sencillo pero poderoso que permite reutilizar código y mantener programas organizados y claros. El ejemplo de la función `saludar()` demuestra cómo se puede definir una función básica y llamar a esta función para ejecutar su tarea en el código principal.

Buenas Prácticas en Nombres de Funciones

Elegir nombres adecuados para las funciones es fundamental para escribir código claro y mantenible. Aquí se presentan algunas recomendaciones clave para nombrar funciones en Python.

Uso de Minúsculas

Los nombres de las funciones deben escribirse en minúsculas. Este estilo sigue la convención de la comunidad Python y mejora la consistencia y legibilidad del código.

```
'''python
def calcular_area():
    pass
```

```
'''
```

Uso de Guiones Bajos

Para separar palabras en los nombres de las funciones, se deben usar guiones bajos (`\_`). Esto ayuda a mejorar la legibilidad y a entender rápidamente el propósito de la función.

```
'''python
def obtener_datos_usuario():
    pass
```

```
'''
```

Nombres Autoexplicativos

Es importante utilizar nombres autoexplicativos que describan claramente lo que hace la función. Evite usar nombres genéricos o abreviaciones que no sean comunes, ya que pueden dificultar la comprensión del código.

```
```python
def calcular_promedio(lista_de_numeros):
 pass

...````
```

Evite nombres como:

```
```python
def cp(l):
    pass

...````
```

En su lugar, use nombres descriptivos que indiquen la acción que realiza la función y los datos con los que trabaja.

Ejemplo Completo

A continuación, se muestra un ejemplo de una función que sigue estas buenas prácticas:

```
```python
def enviar_correo_electronico(destinatario, asunto, mensaje):
 """
 Envía un correo electrónico al destinatario con el asunto y mensaje proporcionados.
 """
 pass

...````
```

En este ejemplo:

- El nombre de la función está en minúsculas.
  - Las palabras están separadas por guiones bajos para mejorar la legibilidad.
  - El nombre de la función es autoexplicativo y describe claramente su propósito.
-

# Conclusión

Adherirse a estas buenas prácticas al nombrar funciones en Python mejora la claridad y mantenibilidad del código. Usar minúsculas, separar palabras con guiones bajos y elegir nombres autoexplicativos facilita la colaboración y el mantenimiento del código a largo plazo.

---

## Variables Locales y Globales

En Python, las variables pueden tener un alcance local o global, lo que determina dónde se pueden acceder y modificar. Comprender la diferencia entre variables locales y globales es crucial para escribir código claro y evitar errores inesperados.

### Variables Locales

Las variables locales son aquellas que se definen dentro de una función y solo son accesibles dentro de esa función. No se puede acceder a ellas desde fuera de la función en la que fueron creadas.

#### Ejemplo de variable local:

```
```python
def ejemplo_local():
    variable_local = 10
    print(variable_local)

ejemplo_local()
print(variable_local) # Esto causará un error

```
```

En este ejemplo, `variable\_local` solo es accesible dentro de la función `ejemplo\_local()`. Intentar acceder a `variable\_local` fuera de la función resultará en un error `NameError` porque la variable no está definida en el ámbito global.

---

### Variables Globales

Las variables globales son aquellas que se definen fuera de cualquier función y son accesibles desde cualquier parte del código, incluidas las funciones.

#### Ejemplo de variable global:

```
```python
```

```
variable_global = 20

def ejemplo_global():
    print(variable_global)

ejemplo_global()
print(variable_global)

'''
```

En este ejemplo, `variable\_global` es accesible tanto dentro como fuera de la función `ejemplo\_global()`.

Uso de Variables Globales y Locales en Funciones

Es posible modificar variables globales dentro de una función utilizando la palabra clave `global`. Sin embargo, se recomienda evitar esto para mantener un código claro y evitar efectos secundarios inesperados.

Ejemplo de modificación de variable global:

```
'''python
variable_global = 20

def modificar_global():
    global variable_global
    variable_global = 30

modificar_global()
print(variable_global) # Imprimirá 30

'''
```

Sin el uso de la palabra clave `global`, una nueva variable local con el mismo nombre se crearía dentro de la función, dejando la variable global sin cambios.

Ejemplo sin modificar la variable global:

```
'''python
variable_global = 20

def intentar_modificar_global():
    variable_global = 30 # Esto crea una nueva variable local
    print(variable_global) # Imprimirá 30

intentar_modificar_global()
```

```
print(variable_global) # Imprimirá 20
```

```
'''
```

En este caso, `variable\_global` dentro de la función es una variable local y no afecta la `variable\_global` global.

Conclusión

Diferenciar entre variables locales y globales es esencial para evitar errores y escribir código más claro y mantenible. Las variables locales se limitan al ámbito de la función en la que se definen, mientras que las variables globales son accesibles en todo el programa. Usar la palabra clave `global` permite modificar variables globales dentro de funciones, pero debe hacerse con precaución para mantener la claridad y evitar efectos secundarios no deseados.

Retornando Valores

La sentencia `return` en Python se utiliza para devolver valores desde una función al llamador de dicha función. Esta capacidad es esencial para que las funciones puedan comunicar resultados y colaborar en la ejecución de un programa más amplio. A continuación, se explicará el uso de `return` y se proporcionará un ejemplo práctico de una función que devuelve un mensaje de saludo personalizado.

Uso de la Sentencia `return`

Cuando una función llega a una sentencia `return`, se detiene inmediatamente y devuelve el valor especificado al llamador de la función. Si no se especifica un valor, la función devuelve `None` por defecto.

Sintaxis básica de `return`:

```
'''python
def nombre_de_la_funcion():
    # Código de la función
    return valor_a_devolver
'''
```

- `return` es la palabra clave utilizada para devolver un valor.
- `valor\_a\_devolver` es el valor que la función devuelve. Puede ser cualquier tipo de dato, como números, cadenas, listas, etc.

Ejemplo Práctico

A continuación, se presenta un ejemplo de una función que utiliza `return` para devolver un mensaje de saludo personalizado:

```
```python
def saludar_con_nombre(nombre):
 mensaje = f'Hola {nombre}! ¿Cómo estás?'
 return mensaje
```
```

En esta función:

- `def` define una nueva función llamada `saludar\_con\_nombre`.
- `nombre` es un parámetro que la función recibe.
- `mensaje` es una variable local que almacena el mensaje de saludo.
- `return mensaje` devuelve el contenido de `mensaje` al llamador de la función.

Llamando a la función y utilizando el valor retornado:

```
```python
resultado = saludar_con_nombre('Juan')
print(resultado)
```
```

Esto imprimirá:

```
```
Hola Juan! ¿Cómo estás?
```
```

Detalles Importantes

1. Terminación de la Función: La función termina su ejecución cuando encuentra una sentencia `return`. Cualquier código escrito después de `return` no se ejecutará.

```
```python
def saludar_con_nombre(nombre):
 mensaje = f'Hola {nombre}! ¿Cómo estás?'
 return mensaje
 # Este código nunca se ejecutará
```

```
print('Esto no se ejecutará')
'''
```

**2. Devolviendo Múltiples Valores:** Python permite devolver múltiples valores separados por comas. Estos valores se devuelven como una tupla.

```
```python  
def obtener_datos():  
    nombre = "Juan"  
    edad = 30  
    return nombre, edad  
  
nombre, edad = obtener_datos()  
print(nombre) # Imprimirá "Juan"  
print(edad)   # Imprimirá 30  
'''
```

Conclusión

El uso de la sentencia `return` es fundamental para que las funciones en Python puedan devolver resultados y colaborar efectivamente dentro de un programa. Comprender cómo y cuándo usar `return` permite escribir funciones más útiles y reutilizables, facilitando la creación de programas más complejos y organizados.

Enviando Valores a una Función

En Python, las funciones pueden recibir datos a través de argumentos. Los argumentos son valores que se pasan a la función cuando se llama. Estos valores se utilizan dentro de la función para realizar operaciones específicas. A continuación, se explica cómo pasar argumentos a una función, ilustrado con un ejemplo práctico de una función `suma()` que recibe dos números y devuelve su suma.

Pasando Argumentos a una Función

Para enviar valores a una función, se deben definir parámetros en la declaración de la función. Los parámetros actúan como variables locales dentro de la función y se les asignan los valores proporcionados durante la llamada a la función.

Definición de una función con parámetros:

```
```python
def suma(numero1, numero2):
 return numero1 + numero2

```
```

En este ejemplo:

- ``def`` es la palabra clave que indica que se está definiendo una función.
 - ``suma`` es el nombre de la función.
 - ``numero1`` y ``numero2`` son los parámetros de la función. Estos parámetros recibirán los valores que se pasen a la función cuando se llame.
-

Llamada a la Función con Argumentos

Cuando se llama a la función, se deben proporcionar los valores correspondientes a los parámetros definidos. Estos valores se conocen como argumentos.

Llamada a la función ``suma()``:

```
```python
resultado = suma(7, 5)
print(resultado)

```
```

Esto imprimirá:

```
```
12
```
```

En este ejemplo:

- ``suma(7, 5)`` llama a la función ``suma`` con los argumentos ``7`` y ``5``.
 - ``numero1`` recibe el valor ``7`` y ``numero2`` recibe el valor ``5``.
 - La función realiza la operación ``7 + 5`` y devuelve el resultado ``12``.
 - El resultado se almacena en la variable ``resultado`` y se imprime en la consola.
-

Orden de los Argumentos

Es importante pasar los argumentos en el mismo orden en que se definen los parámetros. De lo contrario, los resultados pueden no ser los esperados.

Ejemplo de desorden de argumentos:

```
```python
def resta(a, b):
 return a - b

resultado = resta(10, 5)
print(resultado) # Imprimirá 5

resultado = resta(5, 10)
print(resultado) # Imprimirá -5

```
```

En este ejemplo:

- En la primera llamada, `a` recibe `10` y `b` recibe `5`, resultando en $10 - 5 = 5$.
- En la segunda llamada, `a` recibe `5` y `b` recibe `10`, resultando en $5 - 10 = -5$.

Conclusión

Pasar argumentos a las funciones permite que las funciones sean más dinámicas y útiles. Definir parámetros en la declaración de la función y proporcionar argumentos al llamar a la función es esencial para realizar operaciones específicas con diferentes datos. Comprender cómo funcionan los parámetros y argumentos ayuda a escribir funciones más flexibles y reutilizables.

Parámetros y Argumentos

En Python, los términos "parámetros" y "argumentos" se utilizan frecuentemente al trabajar con funciones. Es importante entender la diferencia entre ambos y cómo se utilizan en la programación. A continuación, se detalla la diferencia entre parámetros y argumentos, y se incluyen ejemplos de cómo los argumentos se pueden pasar por posición y por nombre.

Diferencia entre Parámetros y Argumentos

- **Parámetros:** Son variables que se definen en la declaración de una función. Actúan como marcadores de posición para los valores que se pasarán a la función cuando se llame.

```
```python
def sumar(a, b):
 return a + b
```
```

En este ejemplo, `a` y `b` son parámetros de la función `sumar`.

- **Argumentos:** Son los valores reales que se pasan a la función cuando se llama. Los argumentos reemplazan a los parámetros en el momento de la ejecución de la función.

```
```python
resultado = sumar(5, 3)
```
```

Aquí, `5` y `3` son argumentos que se pasan a la función `sumar`.

Pasando Argumentos por Posición

Los argumentos se pueden pasar a una función por posición, es decir, en el mismo orden en que los parámetros fueron definidos.

Ejemplo de Argumentos por Posición:

```
```python
def resta(a, b):
 return a - b

resultado = resta(10, 5)
print(resultado) # Imprimirá 5
```
```

En este ejemplo:

- `a` recibe el valor `10` y `b` recibe el valor `5`.
- La función `resta` realiza la operación `10 - 5` y devuelve `5`.

Si se cambia el orden de los argumentos, los resultados cambiarán:

```
```python
resultado = resta(5, 10)
print(resultado) # Imprimirá -5
```
```

Pasando Argumentos por Nombre

Los argumentos también se pueden pasar a una función por nombre, lo que permite especificar explícitamente a qué parámetro corresponde cada argumento.

Ejemplo de Argumentos por Nombre:

```
```python
def resta(a, b):
 return a - b

resultado = resta(b=5, a=10)
print(resultado) # Imprimirá 5
```
```

En este ejemplo:

- `a` recibe el valor `10` y `b` recibe el valor `5`, independientemente del orden en que se pasaron los argumentos.
- La función `resta` realiza la operación `10 - 5` y devuelve `5`.

El uso de argumentos por nombre puede mejorar la legibilidad del código y evitar errores que pueden surgir del orden de los argumentos.

Conclusión

Comprender la diferencia entre parámetros y argumentos, así como las formas de pasar argumentos por posición y por nombre, es fundamental para escribir funciones claras y efectivas en Python. Los parámetros actúan como variables de marcador de posición dentro de la función, mientras que los argumentos son los valores reales que se pasan a la función cuando se llama.

Funciones Recursivas

La recursividad es una técnica de programación en la que una función se llama a sí misma para resolver subproblemas de un problema más grande. Es especialmente útil para problemas que pueden dividirse en partes más pequeñas del mismo tipo. En Python, las funciones recursivas son comunes y se utilizan para diversas tareas, desde cálculos matemáticos hasta la manipulación de estructuras de datos.

Introducción a la Recursividad

Cuando una función se llama a sí misma, esta llamada se denomina recursión. Una función recursiva generalmente tiene dos componentes esenciales:

1. **Caso Base:** Una condición que detiene la recursión para evitar que la función se llame indefinidamente.
2. **Llamada Recursiva:** La llamada a la propia función con argumentos modificados para acercarse al caso base.

Sintaxis básica de una función recursiva:

```
```python
def funcion_recursiva(parametros):
 if condicion_base:
 # Caso base
 return resultado
 else:
 # Llamada recursiva
 return funcion_recursiva(nuevos_parametros)
...
```
```

Ejemplo de Función Recursiva Sin Retorno

Una función recursiva sin retorno es aquella que no devuelve un valor, sino que simplemente realiza una acción repetida hasta alcanzar el caso base. Un ejemplo común es una cuenta regresiva.

Ejemplo de una cuenta regresiva:

```
```python
def cuenta_regresiva(n):
 if n <= 0:
 print('¡Despegue!')
 else:
 print(n)
```
```

```
    cuenta_regresiva(n - 1)
'''
```

Llamada a la función:

```
```python
cuenta_regresiva(5)
'''
```

Salida:

```
'''
5
4
3
2
1
¡Despegue!
'''
```

En este ejemplo:

- `cuenta_regresiva(n)` se llama a sí misma con `n - 1` hasta que `n` es 0 o menor, lo que constituye el caso base y detiene la recursión.

---

## Ejemplo de Función Recursiva Con Retorno

Una función recursiva con retorno devuelve un valor basado en sus llamadas recursivas. Un ejemplo clásico es el cálculo del factorial de un número.

**Ejemplo de una función factorial:**

```
```python
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
'''
```

Llamada a la función:

```
```python
resultado = factorial(5)
print(resultado)
```

```
```
```

Salida:

```
```
```

```
120
```

```
```
```

En este ejemplo:

- `factorial(n)` se llama a sí misma con `n - 1` hasta que `n` es 0, el caso base, que devuelve 1.
- Cada llamada recursiva multiplica `n` por el resultado de `factorial(n - 1)` hasta que todas las llamadas se completan y se devuelve el valor total.

Conclusión

La recursividad es una herramienta poderosa en la programación que permite resolver problemas complejos de manera elegante y eficiente. Comprender cómo escribir funciones recursivas, tanto con como sin retorno de valores, es crucial para aprovechar al máximo esta técnica.

Funciones Integradas en Python

Python ofrece una amplia variedad de funciones integradas que simplifican muchas tareas comunes de programación. A continuación, se describen algunas de las funciones integradas más utilizadas, junto con ejemplos prácticos de su uso: `int()`, `float()`, `str()` y `round()`.

Función `int()`

La función `int()` se utiliza para convertir un valor a un número entero. Puede convertir valores de otros tipos numéricos, como flotantes y cadenas de caracteres que representen números enteros.

Ejemplo de uso de `int()`:

```
```python
Convertir un flotante a entero
numero_entero = int(2.5)
print(numero_entero) # Imprimirá 2

Convertir una cadena de caracteres a entero
numero_entero = int("25")
print(numero_entero) # Imprimirá 25

```
```

Es importante notar que si la cadena contiene caracteres no numéricos, `int()` lanzará una excepción `ValueError`.

```
```python

Esto generará un error
numero_entero = int("2.5") # ValueError: invalid literal for int() with base 10: '2.5'

```
```

Función `float()`

La función `float()` convierte un valor a un número de coma flotante. Puede convertir números enteros, cadenas que representan números flotantes y otros tipos numéricos.

Ejemplo de uso de `float()`:

```
```python

Convertir un entero a flotante
numero_flotante = float(2)
print(numero_flotante) # Imprimirá 2.0

Convertir una cadena de caracteres a flotante
numero_flotante = float("2.5")
print(numero_flotante) # Imprimirá 2.5

```
```

Función `str()`

La función `str()` convierte un valor a una cadena de caracteres. Puede convertir números, booleanos y otros tipos de datos a sus representaciones en cadena.

Ejemplo de uso de `str()`:

```
```python

Convertir un número a cadena
cadena = str(2.5)
print(cadena) # Imprimirá '2.5'

Convertir un booleano a cadena
cadena = str(True)
print(cadena) # Imprimirá 'True'

```
```

Función `round()`

La función `round()` redondea un número flotante a un número especificado de dígitos decimales. Si no se especifica el número de dígitos, redondea al entero más cercano.

Ejemplo de uso de `round()`:

```
```python

Redondear un número flotante al entero más cercano
numero_redondeado = round(2.6)
print(numero_redondeado) # Imprimirá 3

Redondear un número flotante a un dígito decimal
numero_redondeado = round(2.456, 2)
print(numero_redondeado) # Imprimirá 2.46

```
```

Conclusión

Las funciones integradas de Python como `int()`, `float()`, `str()` y `round()` son herramientas esenciales que facilitan la conversión y manipulación de datos. Comprender cómo utilizar estas funciones permite escribir código más eficiente y legible.